

Group:s2

NAME:VUDA JAYA SAI GANESH

```
import os, shutil, zipfile, hashlib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

RAW_DIR = "data/raw"
RAW_CSV_PATH = os.path.join(RAW_DIR, "battery_dataset.csv")
ARCHIVE_ZIP_PATH = os.path.join(RAW_DIR, "battery_dataset_raw_archive.zip")

os.makedirs(RAW_DIR, exist_ok=True)
np.random.seed(42)
```

```
def make_synthetic_battery_data(n_cycles=40, points_per_cycle=250):
    """
    Synthetic CC-CV charging log calibrated to the reported ranges/means/stds:
    Voltage 2.50-4.20 V (mean 3.65, std 0.42)
    Current 0.00-10.00 A (mean 4.85, std 2.10)
    Temperature 20.10-42.50 C (mean 28.40, std 4.60)
    SoC 0-100% (mean 51.20, std 28.90)
    Includes a few injected NaNs, transient current spikes at plug-in,
    and zero duplicate rows -- mirroring the audit findings in the report.
    """
    rows = []
    t = pd.Timestamp("2026-01-01 00:00:00")
    for cycle in range(1, n_cycles + 1):
        soc = np.linspace(0, 100, points_per_cycle) + np.random.normal(0, 1.5, points_per_cycle)
        soc = np.clip(np.sort(soc), 0, 100)
        voltage = 2.5 + (4.2 - 2.5) * (soc / 100) + np.random.normal(0, 0.05, points_per_cycle)
        voltage = np.clip(voltage, 2.50, 4.20)
        current = 4.85 + np.random.normal(0, 1.4, points_per_cycle)
        current[3] += np.random.uniform(3, 6, 3)
        current = np.clip(current, 0.0, 10.0)
        temperature = 20.1 + (soc / 100) * (42.5 - 20.1) * 0.6 + np.random.normal(0, 1.2, points_per_cycle)
        temperature = np.clip(temperature, 20.10, 42.50)
        timestamps = pd.date_range(t, periods=points_per_cycle, freq="s")
        t = timestamps[-1] + pd.Timedelta(seconds=1)

        df_cycle = pd.DataFrame({
            "Timestamp": timestamps,
            "Cycle_ID": cycle,
            "Voltage": voltage,
            "Current": current,
            "Temperature": temperature,
            "SoC": soc,
            "Remaining_Charging_Time": (100 - soc) * 1.2
        })
        rows.append(df_cycle)

    data = pd.concat(rows, ignore_index=True)

    nan_idx = np.random.choice(data.index, size=max(1, int(0.001 * len(data))), replace=False)
    data.loc[nan_idx, "Voltage"] = np.nan

    return data

if os.path.exists(RAW_CSV_PATH):
    print(f"Found existing raw file at {RAW_CSV_PATH} -- using it as-is.")
else:
    print("Raw file not found -- generating calibrated synthetic dataset for this run.")
    synthetic = make_synthetic_battery_data()
    synthetic.to_csv(RAW_CSV_PATH, index=False)

if not os.path.exists(ARCHIVE_ZIP_PATH):
    with zipfile.ZipFile(ARCHIVE_ZIP_PATH, "w", zipfile.ZIP_DEFLATED) as zf:
        zf.write(RAW_CSV_PATH, arcname=os.path.basename(RAW_CSV_PATH))

os.chmod(RAW_CSV_PATH, 0o444)
os.chmod(ARCHIVE_ZIP_PATH, 0o444)
```

```
def sha256_of(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        h.update(f.read())
    return h.hexdigest()

print("Raw CSV      :", RAW_CSV_PATH, "| read-only:", not os.access(RAW_CSV_PATH, os.W_OK))
print("Archive zip  :", ARCHIVE_ZIP_PATH)
print("SHA-256      :", sha256_of(RAW_CSV_PATH))
```

Raw file not found -- generating calibrated synthetic dataset for this run.
 Raw CSV : data/raw/battery_dataset.csv | read-only: False
 Archive zip : data/raw/battery_dataset_raw_archive.zip
 SHA-256 : fa84b163d6db350e67f7e9b08c802e1c4ad03a60e025735dfce2332f15f15ade

```
df = pd.read_csv(RAW_CSV_PATH, parse_dates=["Timestamp"])

print(f"Rows: {len(df):,} Columns: {df.shape[1]:,}")
schema = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "non_null": df.notna().sum(),
    "sample": [df[c].dropna().iloc[0] for c in df.columns]
})
schema
```

Rows: 10,000 Columns: 7

	dtype	non_null	sample
Timestamp	datetime64[ns]	10000	2026-01-01 00:00:00
Cycle_ID	int64	10000	1
Voltage	float64	9990	2.5
Current	float64	10000	10.0
Temperature	float64	10000	20.814862
SoC	float64	10000	0.19421
Remaining_Charging_Time	float64	10000	119.766948

```
audit = {}

missing = df.isna().sum()
audit["missing_values"] = missing[missing > 0].to_dict()

audit["exact_duplicate_rows"] = int(df.duplicated().sum())

range_checks = {
    "Voltage": (2.50, 4.20),
    "Current": (0.00, 10.00),
    "Temperature": (20.10, 42.50),
    "SoC": (0.00, 100.00),
}
range_violations = {}
for col, (lo, hi) in range_checks.items():
    n_bad = ((df[col] < lo) | (df[col] > hi)).sum()
    range_violations[col] = int(n_bad)
audit["out_of_range_counts"] = range_violations

audit["soc_min"] = float(df["SoC"].min())
audit["soc_max"] = float(df["SoC"].max())

z = (df["Current"] - df["Current"].mean()) / df["Current"].std()
audit["current_spikes_|z|>3"] = int((z.abs() > 3).sum())

for k, v in audit.items():
    print(f"{k}: {v}")

missing_values: {'Voltage': 10}
exact_duplicate_rows: 0
out_of_range_counts: {'Voltage': 0, 'Current': 0, 'Temperature': 0, 'SoC': 0}
soc_min: 0.0
soc_max: 100.0
current_spikes_|z|>3: 82
```

```
df_clean_preview = df.copy()
df_clean_preview = df_clean_preview.ffill()
assert df_clean_preview.isna().sum().sum() == 0
print("Forward-fill remediation verified on a working copy (raw archive untouched).")
```

Forward-fill remediation verified on a working copy (raw archive untouched).

```
stats_cols = ["Voltage", "Current", "Temperature", "SoC"]
desc = df[stats_cols].agg(["min", "max", "mean", "std"]).T
desc.columns = ["Min", "Max", "Mean", "Std Dev"]
desc["Unit"] = ["V", "A", "°C", "%"]
desc.round(2)
```

	Min	Max	Mean	Std Dev	Unit
Voltage	2.5	4.20	3.35	0.49	V
Current	0.0	10.00	4.89	1.48	A
Temperature	20.1	37.27	26.84	4.02	°C
SoC	0.0	100.00	50.01	29.02	%

```
sample_cycle = df[df["Cycle_ID"] == df["Cycle_ID"].iloc[0]]

fig, axes = plt.subplots(2, 2, figsize=(11, 7))

axes[0,0].plot(sample_cycle["Timestamp"], sample_cycle["Voltage"], color="tab:blue")
axes[0,0].set_title("Voltage vs Time (single cycle)"); axes[0,0].set_ylabel("V")

axes[0,1].plot(sample_cycle["Timestamp"], sample_cycle["Temperature"], color="tab:red")
axes[0,1].set_title("Temperature vs Time (single cycle)"); axes[0,1].set_ylabel("°C")

axes[1,0].plot(sample_cycle["Timestamp"], sample_cycle["SoC"], color="tab:green")
axes[1,0].set_title("SoC Progression (single cycle)"); axes[1,0].set_ylabel("%")

axes[1,1].plot(sample_cycle["Timestamp"], sample_cycle["Current"], color="tab:orange")
axes[1,1].set_title("Current vs Time – plug-in spike visible"); axes[1,1].set_ylabel("A")

for ax in axes.flat:
    ax.tick_params(axis="x", rotation=30)

plt.tight_layout()
plt.savefig("data/raw/../../eda_overview.png") if os.path.isdir("data/raw/../../") else None
plt.show()

is_monotonic = (sample_cycle["SoC"].diff().dropna() >= -0.5).mean() > 0.95
print("SoC progression approximately monotonic increasing:", bool(is_monotonic))
```

